PARCIAL PRIMER CUATRIMESTRE 2024 - DISEÑO DE SOFTWARE

TEMA 1

1. En GRASP, el principio de **INDIRECICON** permite evitar el acoplamiento directo entre dos objetos, para desacoplar los objetos se utiliza un intermediario.
2. ¿Qué es diseño? Escriba los conceptos de diseño

El diseño es donde mas calidad agregamos a la solución software, donde la imaginación y la creatividad se mezclan con los requisitos para construir un software que siga los conceptos del buen diseño; modularidad, ocultamiento de la información, refinamiento, abstracción, independencia funcional, bajo acoplamiento, arquitectura, separación de incumbencias y alta cohesión.

"El diseño es lo que casi todo ingeniero quiere hacer. Es el lugar en el que las reglas de la creatividad —los requerimientos de los participantes, las necesidades del negocio y las consideraciones técnicas— se unen para formular un producto o sistema."

El diseño crea una representación o modelo del software, pero, a diferencia del modelo de los requerimientos (que se centra en describir los datos que se necesitan, la función y el comportamiento), el modelo de diseño proporciona detalles sobre arquitectura del software, estructuras de datos, interfaces y componentes que se necesitan para implementar el sistema.

El diseño permite modelar el sistema o producto que se va a construir.
Permite medir la calidad y su mejora antes de generar código.
Permite establecer la calidad del Software

Conceptos de diseño

- Abstracción
- Arquitectura
- Atributos de la calidad
- Buen diseño
- Cohesión
- Diseño de datos
- Diseño del software
- Diseño orientado a objeto, clase, objetos, herencia , mensajes, etc.
- División de problemas
- Independencia funcional
- Lineamientos de la calidad
- Modularidad
- Ocultamiento de información
- Patrones
- Proceso de diseño
- Rediseño
- Refinamiento

3. Verdadero o Falso: Justifique

Los representantes (stubs) no se utilizan en pruebas de integración de sistemas OO porque todas las clases colaboradoras deben estar completamente implementadas.

Falso. El uso de representantes tambien cambia cuando se realiza la prueba de integración en los sistemas OO. Los representantes (stubs) pueden usarse en situaciones donde se requiere la colaboración entre clases, pero donde una o mas de las clases colaboradoras todavia no se implementan por completo.

17.4.2 Prueba de integración en el contexto OO

Puesto que el software orientado a objeto no tiene una estructura de control jerárquico obvia, las estrategias tradicionales descendente y ascendente (sección 17.3.2) tienen poco significado. Además, con frecuencia es imposible integrar las operaciones una a la vez en una clase (el enfoque de integración incremental convencional) debido a las "interacciones directa e indirecta de los componentes que constituyen la clase" [Ber93].

PUNTO CLAVE

Una importante estrategia para la prueba de integración del software OO es la prueba basada en hebra. Las hebras son conjuntos de clases que responden a una entrada o evento. Las pruebas basadas en uso se enfocan en clases que no colaboran fuertemente con otras clases.

Existen dos estrategias diferentes para la prueba de integración de los sistemas OO [Bin94b]. La primera, la *prueba basada en hebra*, integra el conjunto de clases requeridas para responder a una entrada o evento para el sistema. Cada hebra se integra y prueba de manera individual. La prueba de regresión se aplica para asegurar que no ocurran efectos colaterales. El segundo enfoque de integración, la *prueba basada en uso*, comienza la construcción del sistema al probar dichas clases (llamadas *clases independientes*) que usan muy pocas clases *servidor* (si es que usan alguna). Después de probar las clases independientes, se prueba la siguiente capa de clases, llamadas *dependientes*, que usan las clases independientes. Esta secuencia de probar capas de clases dependientes continúa hasta que se construye todo el sistema.

El uso de controladores y representantes también cambia cuando se realiza la prueba de integración de los sistemas OO. Los controladores pueden usarse para probar operaciones en el nivel más bajo, y para la prueba de todos los grupos de clases. También puede usarse un controlador para sustituir la interfaz de usuario, de modo que las pruebas de funcionalidad del sistema puedan realizarse antes de la implementación de la interfaz. Los representantes (*stubs*) pueden usarse en situaciones donde se requiere la colaboración entre clases pero donde una o más de las clases colaboradoras todavía no se implementan por completo.

La *prueba de grupo* es un paso en la prueba de integración del software OO. Aquí, un grupo de clases colaboradoras (determinadas al examinar el CRC y el modelo objeto relacional) se ejercita al diseñar casos de prueba que intentan descubrir errores en las colaboraciones.

4. Verdadero o Falso: Justifique

El refinamiento stepwise es una estrategia de diseño que implica la creación de una jerarquía mediante la descomposición de un enunciado macroscópico de una función en detalles sucesivos hasta llegar a los comandos del lenguaje de programación.

Verdadero. La estrategia plantea separar el enunciado inicial en partes mas pequeñas que se van desarrollando hasta llegar al codigo.

8.3.8 Refinamiento



Existe la tendencia a pasar de inmediato a los detalles e ignorar los pasos del refinamiento. Esto genera errores y hace que el diseño sea mucho más difícil de revisar. Realice refinamiento stepwise.

El refinamiento stepwise es una estrategia de diseño propuesta originalmente por Niklaus Wirth [Wir71]. Un programa se elabora por medio del refinamiento sucesivo de los detalles del procedimiento. Se desarrolla una jerarquía con la descomposición de un enunciado macroscópico de la función (abstracción del procedimiento) en forma escalonada hasta llegar a los comandos del lenguaje de programación.

En realidad, el refinamiento es un proceso de *elaboración*. Se comienza con un enunciado de la función (o descripción de la información), definida en un nivel de abstracción alto. Es decir, el enunciado describe la función o información de manera conceptual, pero no dice nada sobre los trabajos internos de la función o de la estructura interna de la información. Después se trabaja sobre el enunciado original, dando más y más detalles conforme tiene lugar el refinamiento (elaboración) sucesivo.

La abstracción y el refinamiento son conceptos complementarios. La primera permite especificar internamente el procedimiento y los datos, pero elimina la necesidad de que los "extraños" conozcan los detalles de bajo nivel. El refinamiento ayuda a revelar estos detalles a medida que avanza el diseño. Ambos conceptos permiten crear un modelo completo del diseño conforme éste evoluciona.

TEMA 2

CONCEPTOS TEORICOS

a) ¿Qué son las Dimensiones Garvin? ¿Y cuáles son?

Dimensiones de la calidad de Garvin

Garvin sugiere que la calidad debe tomarse en cuenta, adoptando un punto de vista multidimensional

1. Desempeño.
2. Características.
3. Confiabilidad
4. Conformidad
5. Durabilidad.
6. Servicio.
7. Estética.
8. Percepción.



b) Verdadero o Falso: Justifique

En el software orientado a objetos, las estrategias tradicionales de prueba de integración descendente y ascendente son altamente efectivas debido a la estructura jerárquica obvia.

Falso. Para el software orientado a objetos, las estrategias recomendadas son las pruebas en hebras y las basadas en uso. Como cada clase se relaciona con otras, es muy complejo encontrar una estructura jerárquica obvia.

Falso. En el software orientado a objetos, la estructura de las relaciones entre clases no siempre es jerárquica de manera obvia. A menudo, las clases están interrelacionadas mediante composición, herencia y asociaciones diversas, lo que puede complicar la aplicación de estrategias tradicionales de integración descendente y ascendente. En estos entornos, las estrategias de pruebas en hebras y basadas en uso son más adecuadas:

c) Verdadero o Falso: Justifique

La abstracción y el refinamiento son conceptos opuestos y no pueden ser utilizados conjuntamente en el proceso de diseño.

Falso. No son conceptos opuestos, si no complementarios. Ambos conceptos permiten crear un modelo completo del diseño conforme este evoluciona.

La abstracción y el refinamiento son conceptos complementarios. La primera permite especificar internamente el procedimiento y los datos, pero elimina la necesidad de que los "extraños" conozcan los detalles de bajo nivel. El refinamiento ayuda a revelar estos detalles a medida que avanza el diseño. Ambos conceptos permiten crear un modelo completo del diseño conforme éste evoluciona.

d) Seleccionar los principios generales de diseño de interfaces webapps:

- Previsión
- Ley de Fitt
- Reducción de la latencia
- Atajos intuitivos
- Autonomía Controlada
- Objetos de interfaz humana
- Navegación Visible
- Todos los objetos tienen interacción

Principios básicos del diseño de interfaz Web

A fin de diseñar interfaces de webapps con estas características, Tognozzi [Tog01] identifica un conjunto de principios generales de diseño:

- Previsión.
- Comunicación.
- Consistencia.
- Autonomía controlada.
- Eficiencia
- Flexibilidad.
- Centrarse.
- Ley de Fitt.
- Objetos de la interfaz humana.
- Reducción de la latencia.
- Aprendizaje
- Metáforas.
- Mantener la integridad de los productos del trabajo.
- Legibilidad.
- Dar seguimiento al estado.
- Navegación visible.

e) Los elementos más importantes en UX son: User Personas, Pruebas de Usabilidad, HeatMaps, Investigación, Prototipado

Métodos y herramientas de investigación de usuarios

User Personas

Crear personajes ficticios basados en datos del usuario para comprender sus necesidades, comportamiento y expectativas.

Prueba de usabilidad

Observar a los usuarios interactuando con el producto para identificar problemas y determinar soluciones para optimizar la UX.

HeatMaps

Herramientas para obtener información visual sobre cómo los usuarios interactúan con el producto, los patrones de clics y las áreas más visibles de su diseño

PRACTICA

Sistema de Gestión de Turnos Médicos para una Clínica

Se necesita diseñar una solución para gestionar turnos médicos para una clínica

Funcionalidades para Pacientes

Registro y Autenticación de Usuarios: Los pacientes deben registrarse y crear una cuenta ingresando su correo electrónico, numero de celular, edad y obra social.

Información Personal: Datos personales como nombre, apellido, edad, obra social y datos de contacto.

Historial Médico: Registro de antecedentes médicos y preferencias personales de tratamiento.

Reserva de Turnos:

Búsqueda de Disponibilidad: Los pacientes pueden buscar turnos disponibles por médico, especialidad y fecha.

Selección y Confirmación de Turno: Los pacientes pueden seleccionar y confirmar un turno especifico.

Notificaciones: Envío y confirmaciones y recordatorios de turnos por correo electrónico o SMS.

Historial de Turnos: Los pacientes pueden acceder al historial de turnos anteriores, incluyendo detalles de visitas pasadas.

Control de Inasistencias: Si un paciente no asiste a tres turnos confirmados, se le cobrará una consulta y el conteo de inasistencias se reiniciará a cero.

Mensajería: Comunicación segura entre pacientes y médicos para consultas rápidas.

Notificaciones: Notificaciones sobre cambios en el turno, recordatorios de citas y otros eventos importantes.

Funcionalidades para Médicos y Personal de la Clínica

Registro y Autenticación de Usuarios: El sistema tiene un listado preexistente de empleados con información básica (nombre, apellido, rol, correo electrónico). El sistema envía un correo electrónico con un enlace y una contraseña temporal para completar el registro. El sistema cada 3 días valida que los usuarios hayan realizado la finalización de registro*.

Completar Registro* Los empleados deben completar su perfil ingresando información adicional: Los médicos deben definir su especialidad y perfil profesional, mientras que todos los empleados deben registrar su domicilio, teléfonos, horarios de atención, y datos de contacto adicionales (correo electrónico alternativo).

Gestión de Calendario y Horarios de los Médicos: Administración de los horarios de trabajo y disponibilidad de cada médico.

Bloqueo de Horarios: Posibilidad de bloquear horarios para vacaciones, días libres o emergencias.

Mensajería: Comunicación segura entre médicos y pacientes para consultas rápidas.

Notificaciones: Notificaciones sobre cambios en el turno, recordatorios de citas y otros eventos importantes.

Implementar estas funcionalidades asegurara una gestión eficiente de los turnos médicos y una mejor experiencia tanto para los pacientes como para los médicos y el personal administrativo de la clínica.

NOTA: Importante, ante cualquier inquietud de interpretación, utilizar y explicar supuestos sobre el caso.

Se pide:

1. Diagrama de Clases de Diseño COMPLETO
2. En el DC, aplique 2 patron GOF
    a. Implementar un patron GOF que permite que un objeto altere su comportamiento cuando el turno cambie de estado (manejar los diferentes estados de un turno (Disponible, Reservado, Cancelado).)

   b. Implementar un patron GOF que permite encapsular una petición como un objeto, permitiendo parametrizar a los clientes con diferentes solicitudes. (Esto es útil para operaciones como la reserva y cancelación de turnos.)
   c. Implementar un patron GOF que permite y defina una dependencia uno a muchos entre objetos para enviar notificaciones a los pacientes y médicos cuando hay cambios en los turnos.

3. Diagramas de Secuencia para un escenario significativo (escenario especifico a elección del alumno)